Nugget: Portable Program Snippets

By

Zhantong Qiu
M.S. (University of California Davis) 2026

Thesis

Submitted in partial satisfaction of the requirements for the degree of

Master of Science

in

Computer Science

in the

Office of Graduate Studies

of the

University of California

Davis

Approved:

Chair Jason Lowe-Power

John Owens

Tapti Palit

Committee in Charge

2026

Copyright © 2026 by

Zhantong Qiu

All rights reserved.

Contents

List of Figures iv

List of Tables vi

Abstract vii

Acknowledgments viii

1 Introduction **1**

2 Motivation and Prior Work **4**

2.1 Samples Are Expensive To Find 4

2.2 Samples Are Tied To A Single Executable Binary 5

2.3 Validating The Selected Samples Is Infeasible 6

3 The Nugget Framework **8**

3.1 Unit of Work 10

3.2 Preparation 11

3.3 Interval Analysis 12

3.3.1 Interval Analysis LLVM Pass 13

3.3.2 Interval Analysis Executable 13

3.4 Nugget Creation 15

3.4.1 Markers 15

3.4.2 Overhead of Hooks 16

3.5 Sample Validating 17

4 Evaluation **19**

4.1 Interval Analysis Efficiency (Goal 1) 21

4.1.1 Comparison with Functional Simulation 21

4.1.2 Mix of Workload Types 23

4.1.3 Scaling with Number of Threads 24

4.1.4 Summary 26

4.2 Applying Sampling Methodologies to the Nugget Framework and Validating the Selected Samples (Goals 2 & 3) 27
4.2.1 Sampling Methodologies 27
4.2.2 Sample Validation 28
4.2.3 Hook Overhead 29

5 Case Studies **32**
5.1 Nuggets in Simulation and on Hardware 32
5.1.1 Experimental Setup 33
5.1.2 Results 34
5.2 Model Accuracy Evaluation 37

6 Conclusion **39**

A Artifact Appendix **46**
A.1 Abstract 46
A.2 Artifact check-list (meta-information) 46
A.3 Description 48
A.3.1 How to access 48
A.3.2 Hardware dependencies 48
A.3.3 Software dependencies 49
A.3.4 Data sets 49
A.3.5 Models 49
A.4 Installation 49
A.5 Experiment workflow 50
A.6 Evaluation and expected results 51
A.7 Experiment customization 52

List of Figures

3.1 Nugget pipeline. The process is divided into two main stages: (1) preparation and analysis, and (2) nugget selection and validation. The preparation and analysis stage, including base IR file creation and interval analysis execution, only needs to be performed once per program. Nugget creation and validation can be repeated to refine and select the most representative set of samples for the workload. 10

4.1 Interval analysis overhead via Nugget and functional simulation. 22

4.2 Interval analysis overhead via Nugget across different workloads. 23

4.3 Interval analysis overhead via Nugget across different numbers of threads. 25

4.4 Prediction error by machine and sampling method. **All results shown are obtained from executing the nuggets directly on real hardware**, without using simulation. 26

4.5 The number of marker hook executions for the randomly selected nuggets shown in Figure 4.4. The number of hook executions is normalized to the total number of IRBB executions during all nugget executions for each workload and input pair. 30

5.1 Distribution of absolute speedup prediction errors using K-means samples across simulation (blue) and hardware (orange) comparisons for selected benchmarks. "0.1" means 10% difference in predicted speedup. 34

5.2 Speedup: Random vs K-means samples when ISA varies and microarchitecture is fixed, Config 3 vs Config 4. 35

5.3 Speedup between Random and K-means samples when microarchitecture varies and ISA is fixed: Config 4 over Config 2 for Arm, and Config 3 over Config 1 for x86. 36

5.4 Speedup comparison between Random and K-means samples when both ISA and microarchitecture vary across simulation models. “K” indicates K-means samples, and “R” indicates Random samples. 36
5.5 Comparison of the number of memory instructions for two different microcodings in gem5 vs real hardware. 38

List of Tables

4.1 Real Hardware Configuration 20
4.2 Overview of Benchmark Suites Used in Evaluation 20

5.1 gem5 Simulation Model Configuration. The large, small, and base out-of-order cores are internal models that target different performance characteristics. 33

Abstract

Nugget: Portable Program Snippets

Evaluating architectural ideas on realistic workloads is increasingly challenging due to the prohibitive cost of detailed simulation and the lack of portable sampling tools. Existing targeted sampling techniques are often tied to specific binaries, incur significant overhead, and make rapid validation across systems infeasible. To address these limitations, we introduce Nugget, a flexible framework that enables portable sampling across simulators, hardware, architectural differences, and libraries. Nugget leverages LLVM IR to perform binary-independent interval analysis, then generates lightweight, cross-platform executable snippets, *nuggets*, that can be validated natively on real hardware before use in simulation. This approach decouples samples from specific binaries, dramatically reduces analysis overhead, and allows researchers to iterate on sampling methodologies while efficiently validating samples across diverse systems.

Acknowledgments

We thank Alen Sabu, Christopher Batten, Derrick Quinn, Matthew D. Sinclair, Oscar Hernandez, and the Davis Computer Architecture Research Group for feedback used to improve this work. This work was supported by NSF grant 2311888, Department of Energy, Oak Ridge National Laboratory, and Los Alamos National Laboratory.

Chapter 1

Introduction

Evaluating application performance on novel architectural ideas is key to computer architecture research. However, especially for hardware which does not yet exist, evaluating application performance can be very time-consuming. For instance, when using cycle-level software simulation, there can be over 100,000× slowdown. Even using FPGA-accelerated simulation is significantly slower than wall-clock time [27]. At the same time, modern applications are growing in their computation demands from exascale applications [34] to machine learning training (GPT-4 required 2,150,000 Exa- or 2.15×10^{25} operations to complete [38]).

To facilitate the evaluation of large applications, there has been a long history of reducing applications to smaller benchmarks, kernels, or short representative intervals. For instance, Livermore Loops provide a set of microbenchmarks that represent key computational kernels from scientific applications [18]. It is also common to pull out kernels from an application or to create "proxy applications" [4]. While these methods can reduce very large applications to much more manageable benchmarks, they require application experts to manually identify the important parts of applications and create new programs.

Another approach to reducing the size of applications is to *sample* the application to automatically find *representative intervals* of different application phases. Many techniques (e.g., SimPoint [41], BarrierPoint [14], and LoopPoint [39]) follow this approach.

In this paper, we focus on targeted sampling techniques, which use profiling to automatically select samples. Prior targeted sampling techniques have three main drawbacks

that limit their applicability to modern, realistically sized applications:

- **Samples are expensive to find:** Targeted sampling techniques require running the application from beginning to end once (typically via simulation or slow emulation) to define intervals, analyze program phases, and identify representative samples.
- **Samples are tied to a single executable binary:** These techniques generate samples that are specific to a particular binary, so any binary change necessitates repeating the entire sampling process.
- **Validating the selected samples is infeasible:** Targeted sampling techniques require running the application in detailed simulation to verify that the selected samples accurately represent the program.

To address these limitations, we present a framework called *Nugget* for agile targeted sampling development. Because modern applications exhibit varied characteristics, we believe there is no one-size-fits-all sampling methodology or selection algorithm [16,24,28, 51]. Therefore, Nugget is designed to be flexible and extensible, allowing researchers to create new sample selection algorithms that can be tailored to their targeted workload and input without worrying about the current limitations of targeted sampling techniques. Additionally, using Nugget for sample selection methodologies, because of its nature of cross-platform compatibility and its ability to run on real hardware, enables richer insights into the selected samples and the systems that run the samples. The main contributions of this paper are:

- **Efficient and portable interval analysis** on real hardware using LLVM, unconstrained by the hardware's microarchitecture or ISA.
- **An LLVM IR-level unit of work**, enabling sample selection algorithms to create and locate samples for a program in a cross-platform manner, agnostic of machine-level instructions, such as architecture-specific instructions and optimizations.
- **A sample creation methodology** that allows samples to run on any platform (e.g., real hardware or simulators).

- **An efficient validation methodology** that validates the selected samples for the target workload and input on native hardware, integrated into the development workflow.

Chapter 2

Motivation and Prior Work

Nugget is designed to address the following limitations of prior sampling techniques:

- Samples are expensive to find.
- Samples are tied to a single executable binary.
- Validating the selected samples is infeasible.

In this section, we discuss each of these limitations and how prior work in sampling has or has not addressed them.

2.1 Samples Are Expensive To Find

Targeted sampling techniques require analyzing the entire workload execution, which is a time-consuming process.

For example, the SPECspeed suite in the SPEC CPU2017 benchmark suite [11] has an average of 21.8 trillion dynamic instructions per benchmark for floating point benchmarks with reference input size, which is the only input size designed to report time metrics [31]. Functional simulators, such as the gem5 atomic model, are often limited to a simulation rate of approximately 1 Million Instructions Per Second (MIPS) [5]. Therefore, it takes approximately 250 days of wall-clock time to analyze an average SPECspeed benchmark with reference input size. More recent workloads (e.g., the forthcoming SPEC CPUv8 [44],

HPC workloads, and machine learning workloads) are significantly larger, making functional simulation infeasible.

For multi-threaded benchmarks, the analysis overhead is even worse. We must analyze the program once for every different thread configuration because the number of threads affects the program's execution. Also, if the workload undergoes any changes (e.g., new compiler, different optimizations, etc.), we must redo the analysis.

Live sampling tools like Pac-Sim [32] parallelize the phase analysis and detailed simulation to reduce the overall phase analysis penalty. However, live sampling still relies on the simulator to analyze the program's execution so this technique is constrained by the simulation speed. If a benchmark takes the simulator 250 days to analyze, as mentioned in the above example, then parallelizing the analysis and detailed simulation will still take at least 250 days.

Tools like DynamoRIO [10] and the Intel PinPoints toolkit [35] can analyze the program's execution with dynamic binary instrumentation or binary translation. The instrumentation and profiling routines introduce overhead to the program's execution, but this overhead is significantly smaller than that of simulation (e.g., 4-40× for QEMU with some single-threaded workloads [47]). However, these tools often have restrictions on the architectures they can analyze or the host machines. Developing architectures that need to be analyzed with a wider range of benchmarks, such as RISC-V, often cannot use these tools because they are not supported.

Goal 1: Analyze the program's execution quickly and without any restrictions on the architecture or host machines.

2.2 Samples Are Tied To A Single Executable Binary

Most prior work relies on binary-specific information, such as the program counter, to define intervals, identify program phases, and select samples. Any binary change, such as recompiling with different ISA extensions or compilers, alters the static instructions and the dynamic instruction stream, invalidating previously selected samples. As discussed in Section 2.1, the high overhead of this analysis makes repeating it for every binary revision

time-consuming.

Cross-binary simulation points [36] represent a notable attempt to decouple samples from a specific binary. However, they are limited to single-threaded programs and require significant manual effort, including mapping their control flow graphs and recalculating sample weights.

Goal 2: Automatically generate samples that are independent of the binary.

2.3 Validating The Selected Samples Is Infeasible

Sampling methodologies aim to predict overall program performance by measuring only a subset of representative samples and extrapolating the full-program behavior using statistical models. Most methodologies focus on predicting metrics that summarize the entire hardware performance, such as instructions per cycle (IPC) or total runtime.

To evaluate the accuracy of a set of samples selected by a sampling methodology, the predicted performance metrics are compared against ground truth measurements. First, the target performance metric is measured from a full execution of the program on the experimental system, serving as the ground truth. Next, the same metric is measured for the selected samples, and statistical processing is applied to estimate the full-program value. Finally, the predicted value is compared to the ground truth to quantify the accuracy of the selected samples.

The effectiveness of a sample selection method depends on benchmark characteristics. Differences in control flow phase behavior, working set size and locality, and input sensitivity can cause a method that succeeds on one workload and input pair to misrepresent another [12, 15, 16, 19, 41, 48, 51]. For example, empirical studies (e.g., MAV [12]) report outliers where SimPoint fails to capture distinct memory access behavior while SimPoint reports average 3% IPC error with SPEC CPU2000 [42] programs [26]. Hence, representative intervals should be validated. However, the only way to validate them is to run the entire workload.

Prior sampling techniques are typically evaluated on small benchmarks or truncated runs because their validation depends on simulation [14, 22, 26, 30, 32, 36, 39–41, 49, 50].

A missing piece is an agile path from methodology development to real hardware level validation. The intent of the sample set is to approximate the behavior of the real system, so validation by simulation is only meaningful if the simulator faithfully models the target. If it does not, the validation does not transfer. This also leads to difficulty in comparing different methodologies as results are tied to different simulators, forcing redundant engineering effort before we can make apples-to-apples comparisons. For example, LoopPoint is validated with SPEC CPU2017 benchmarks using train-size input with Sniper [13]. FSA [40] is validated with the first 30 billion instructions of the SPEC CPU2006 [43] benchmarks with gem5 [9, 33]. These techniques can only be validated in a limited way because they require a full detailed simulation from beginning to end of the workload to validate. Detailed simulation can be 2–10× slower than functional simulation, which implies that validation of full-sized workloads could take nearly a decade!

Lack of an agile workflow to validate quickly has led to sampling techniques being rarely validated with large benchmarks. Evidence suggests that validation results from smaller benchmarks or partial executions do not generalize to larger benchmarks and realistic workloads [26, 31, 41, 48, 51]. Moreover, the program behavior of a workload with different input can be different [15]. Thus, per benchmark and input pair validation is needed.

Goal 3: Quickly validating the selected samples for the targeted benchmarks and input size.

Chapter 3

The Nugget Framework

Nugget is designed to address the limitations of the prior sampling techniques, specifically focusing on 1) the high cost of finding samples, 2) the coupling of the samples to a specific executable binary, and 3) the infeasibility of validating the selected samples. Nugget leverages LLVM passes to statically insert annotations to analyze the program's execution on real hardware without the need for simulation, emulation, or binary translation. By using LLVM intermediate representation (IR) to define program progress and intervals, the samples become independent of the binary and can be reused across different executable versions of the same program, since LLVM IR abstracts away details of the final binary. Lastly, Nugget provides a method to create portable samples to be executed on any platform, such as real hardware or simulators, so that the samples can be validated in native time with real hardware before running them on a simulator.

In this section, we present an overview of Nugget as a pipeline (Figure 3.1), describing its workflow from preparation through interval analysis, nugget creation, and finally, sample selection verification. It is not the focus of this paper to introduce a new sample selection algorithm, but rather to provide a framework that can be used to implement and validate diverse sample selection methodologies that can be tailored to specific applications and workloads.

We begin by defining program progress and intervals, which ensures that samples remain independent of the binary, as described in Section 3.1. Next, we describe the preparation required by Nugget to guarantee the LLVM IR of the program remains consistent

across different binaries in Section 3.2. Then, we introduce our novel interval analysis methodology, which leverages LLVM passes, enabling comprehensive program analysis directly on real hardware without imposing hardware-specific constraints in Section 3.3. Following that, we present the nugget creation process, which allows for the generation of portable samples that can be executed on any platform, including real hardware and simulators, in Section 3.4. Finally, we describe the sample selection validation process, which enables efficient validation of the selected samples for specific workloads in native time, ensuring that the samples accurately represent the program's execution in Section 3.5.

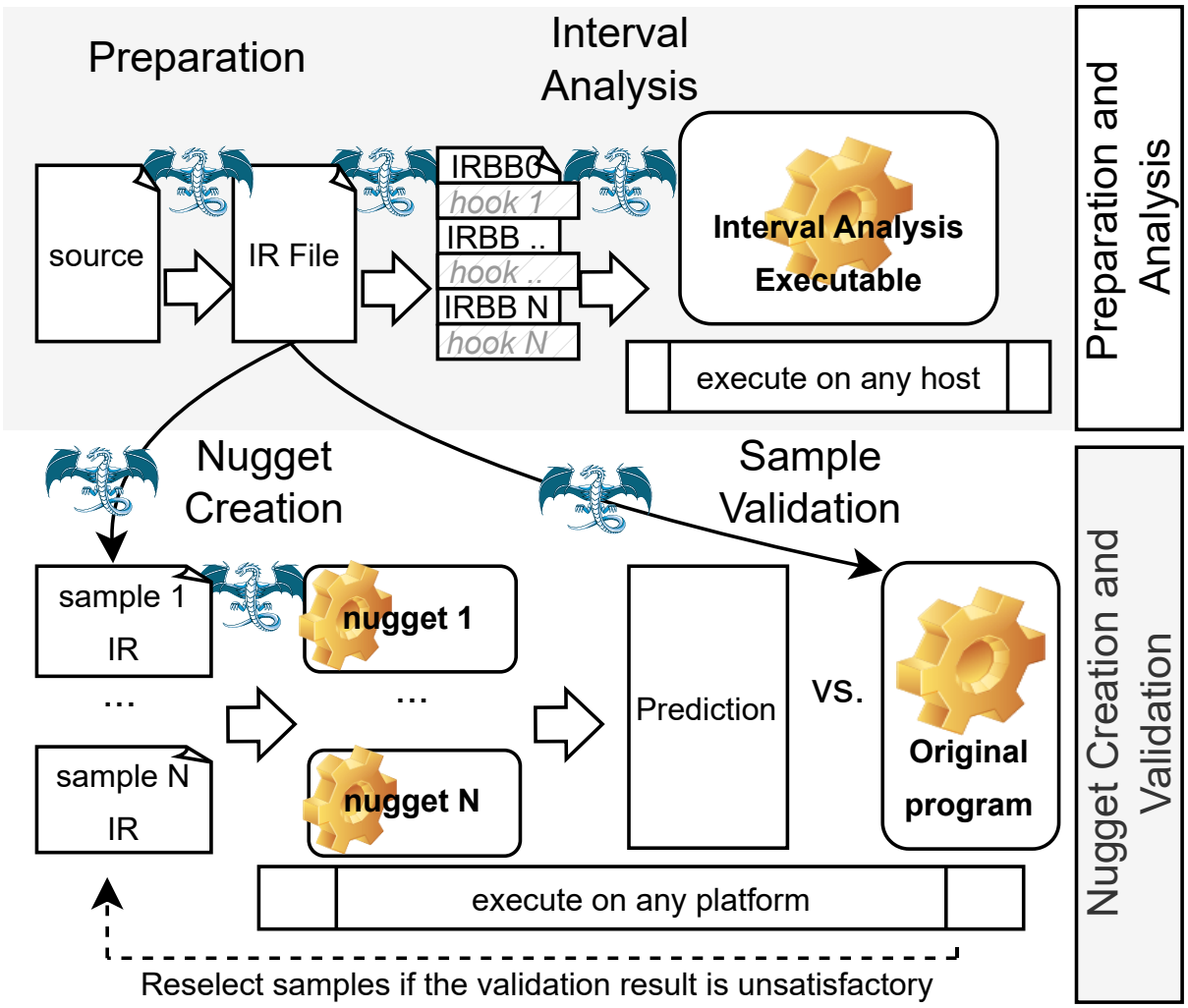

Figure 3.1: Nugget pipeline. The process is divided into two main stages: (1) preparation and analysis, and (2) nugget selection and validation. The preparation and analysis stage, including base IR file creation and interval analysis execution, only needs to be performed once per program. Nugget creation and validation can be repeated to refine and select the most representative set of samples for the workload.

3.1 Unit of Work

A unit of work defines the metric by which progress toward completing a program's execution is measured. It must remain consistent across identical runs of a program with the same input. While machine instructions could serve this purpose for a single binary, Nugget instead counts executed LLVM IR instructions as its unit of work. This ensures a uniform measurement of program progress across all binaries of the same program with the same input.

A unit of work directly determines the notion of intervals. For example, if the unit of work is the number of executed instructions, then each interval corresponds to a fixed number of executed instructions during runtime. A sample is a specific interval of a program's execution; thus, the unit of work also defines what constitutes a sample.

The choice of unit of work influences not only how intervals are defined, but also the sample selection algorithm and the statistical models used in performance prediction. Therefore, the unit of work must be defined consistently across all binaries to ensure that samples remain independent of any specific binary. For example, if the unit of work is the number of executed machine-level instructions, then the sample will be tied to a specific binary because any binary changes can affect the number of machine-level instructions executed, even if the control flow remains the same [36].

LLVM [29] is an open-source compiler infrastructure that transforms source code into a platform-independent intermediate representation (IR) before generating machine-specific binaries. LLVM IR instructions abstract away the underlying machine-level instructions, so as long as the control flow remains the same at the LLVM IR level, the definition of all intervals of program execution remains consistent across different binaries. Since LLVM IR instructions are an abstraction of machine-level instructions, the number of executed LLVM IR instructions reflects the progress of program execution just as the number of executed machine-level instructions does.

Therefore, Nugget's unit of work is suitable for applications that have stable control flow across different platforms. Applications such as gradient descent, where floating-point precision might affect the control flow, are not suitable for Nugget or for any sampling methodologies that rely on the application having stable control flows across microarchitectures [14, 26, 30, 32, 36, 39, 41, 50].

3.2 Preparation

In the preparation stage, we generate the optimized LLVM IR representation of the program, which is used consistently throughout all stages of the pipeline (Figure 3.1). This is because Nugget relies on a stable LLVM IR representation of the program for its unit

of work, as introduced in Section 3.1.

Optimization is important for the program's performance. LLVM IR level optimizations, such as loop unrolling, can significantly affect control flow. To maintain consistency, Nugget applies these optimizations during the generation of the base LLVM IR representation in the preparation stage. In contrast, most backend optimizations, such as replacing scalar instructions with AVX vector instructions, do not alter the structure of the LLVM IR. While certain backend passes (e.g., late-stage loop unrolling) may modify LLVM IR layout under specific conditions, such passes are not enabled by default in LLVM.

While build-time modes such as LTO and PGO can alter LLVM IR, they are disabled by default. For our experiments, the LLVM IR structure remained stable across optimization levels, CPU-feature flags, and LLVM versions. Therefore, we can apply the backend optimization in the final binary compilation safely.

Only code included in this base LLVM IR representation of the program will be subject to analysis. For instance, any linked libraries must also be processed through Nugget during this preparation stage; otherwise, their contents will be excluded from the analysis. This selective inclusion also allows us to ignore irrelevant or non-contributory operations, such as spin loops in multi-threaded libraries [39], that do not meaningfully impact program progress.

3.3 Interval Analysis

Nugget uses LLVM passes to automatically annotate programs at the LLVM IR level, enabling interval analysis to be performed directly on real hardware. Interval analysis is necessary to discover and profile all intervals in a program's execution as defined by the chosen unit of work. However, profiling an entire program can be time-consuming if done via simulation or emulation (Section 2.1).

While tools that rely on dynamic instrumentation and binary translation exist to perform such analyses, they often impose restrictions on the ISA and the host platform. By embedding interval analysis directly into the program's execution, Nugget achieves near-native speed without introducing hardware-specific constraints: any machine capable

of running the program can execute the interval analysis.

Because our unit of work is defined at the LLVM IR level and is independent of the final binary, all discovered intervals remain consistent regardless of the host machine's ISA or micro-architecture. For example, we can analyze a program on an x86 system and then use the resulting intervals to drive sampling on an Arm system.

In this section, we describe (1) how Nugget uses LLVM passes to generate an interval analysis executable, and (2) how this executable is used to discover and profile intervals.

3.3.1 Interval Analysis LLVM Pass

The interval analysis LLVM pass, a modular compiler transformation that operates directly on LLVM IR, identifies all basic blocks (IRBBs) in the program's IR. For each IRBB, it records the number of LLVM IR instructions and inserts a hook at the end of the block. Each hook is uniquely associated with its IRBB and encodes both the block's ID and instruction count, allowing us to retrieve LLVM IR-level information during binary execution.

The hook is a lightweight function that counts occurrences and invokes other functions when specified conditions are met. This design allows us to flexibly adjust what data to collect by simply changing the functions the hook calls. After compilation with this LLVM interval analysis pass, the resulting binary executes the hook every time an IRBB is completed, enabling runtime data collection at the LLVM IRBB level. This hook-based approach also lets us automatically map LLVM IR basic blocks to their corresponding machine-level basic blocks. We discuss the overheads introduced by the hooks in Section 3.4.2.

3.3.2 Interval Analysis Executable

The interval analysis process has two main goals: (1) to partition the program's execution into a sequence of intervals, and (2) to generate a characteristic signature (e.g., a basic block vector) for each interval. In Nugget, an interval is defined by a fixed number of executed LLVM IR instructions (Section 3.1). Thus, discovering intervals involves recording the current execution point whenever the LLVM IR instruction count is reached

and marking the current execution point as the end of a newly identified interval.

Once intervals are identified, each is characterized by a signature. Whereas the interval's definition specifies its location in the program's execution, the signature captures its behavioral characteristics. For example, SimPoint defines intervals by instruction count but represents each interval's signature as a basic block vector that tracks how often each basic block is executed.

A signature is closely tied to the chosen sample selection algorithm. Since this paper does not propose a new selection algorithm, we do not introduce a new signature. However, we demonstrate how our hook-based approach can flexibly collect different types of signatures by constructing both LLVM IRBB vectors and count-stamp vectors. The LLVM IRBB vector tracks the frequency of entering each IRBB, while the count-stamp vector records the global IR instruction counter value when each IRBB is last entered. These will later be used to create markers (Section 3.4).

Discovering Intervals Discovering intervals is managed by the hooks. Each hook knows the ID and instruction count of its IRBB. Whenever a hook executes, it increments a global IR instruction counter by the IRBB's instruction count and checks whether this counter has reached the defined interval size. If so, a new interval is marked by assigning it a sequential ID; otherwise, execution continues as normal.

All discovered intervals are binary independent because their definitions rely on the LLVM IR-level. Thus, regardless of the ISA or microarchitecture of the machine running the interval analysis executable, the discovered intervals remain consistent for the final compiled binary.

Generate A Characteristic Signature Our hook-based design makes it easy to specify what data to collect during execution by simply modifying the functions called by each hook. Because each hook has access to the LLVM IR basic block (IRBB) information, we can build an *IRBB vector* by incrementing a global vector based on the IRBB ID whenever a block is entered. This vector effectively records how many times each IRBB is executed within an interval, capturing the frequency profile of the program's control flow. It is recorded and reset whenever a new interval is detected. Since the LLVM IR

control flow remains consistent across all final binaries (Section 3.2), the IRBB vector is inherently binary independent.

In contrast, the *count-stamp vector* records the global IR instruction count each time an IRBB is entered, marking when during the program's execution each IRBB was last encountered. This provides a timestamped trace of the control flow relative to instruction execution. The count-stamp vector is also recorded and reset at the end of each interval.

In Section 3.4.1, we show how the IRBB vector is used to create execution markers and how the count-stamp vector helps reduce the hook overhead. This flexible design allows researchers to explore additional signatures for sampling methodologies with minimal development effort.

3.4 Nugget Creation

After selecting a set of samples with a sampling algorithm, Nugget generates *nuggets* based on these samples. We define a *nugget* as an executable that marks the selected sample with a start point and an end point, enabling execution on any platform capable of running the original program, including both real hardware and simulators. Thus, every *nugget* requires a cross-platform start marker and end marker to indicate the boundaries of the sample execution. In this section, we describe how Nugget creates these markers and mitigates their overhead.

3.4.1 Markers

In Nugget, a *marker* is a mechanism for identifying a specific point in program execution, defined according to our unit of work. Because progress is measured by the number of executed LLVM IR instructions (Section 3.1), our markers are directly tied to the IR instruction count.

We designate the last executed LLVM IR basic block (IRBB) that completes the target instruction count as a *marker*. This marker consists of two components: the IRBB itself and the number of times it must execute. The end marker for an interval is determined using the IRBB vector and count-stamp vector gathered during interval analysis (Section 3.3.2). Specifically, we use the count-stamp vector (Section 3.3.2) to

identify the last executed IRBB of the interval, which becomes the basic block for the end marker, and the IRBB vector to determine how many times that block must execute to reach the interval's endpoint. Because the start of one interval corresponds to the end of the previous interval, we reuse the end marker of the preceding interval as the start marker for the next.

Nugget inserts hooks at these marker IRBBs during the nugget creation LLVM pass. These hooks track IRBB executions and trigger specific actions, such as resetting statistics in a simulator or starting a timer on real hardware.

When using *nuggets* in simulation, we can also mark the start of a warmup interval. The warmup interval does not belong to the selected sample but is a period of execution used to warm up the microarchitectural state in simulation models, assuming functional simulation is used to fast-forward execution before the sample starts [20]. Any contiguous and antecedent intervals of the sample interval can be used as the warmup interval, so one can choose how many intervals to use for warmup.

3.4.2 Overhead of Hooks

Hooks inserted at marker IRBBs introduce overhead due to frequent counting and condition checks, which can affect measurements if executed often. This overhead appears when running *nuggets* on real hardware, as the hooks add extra instructions.

For single-threaded programs, the overhead is typically small because hooks perform only simple operations, such as counting and checking thresholds. Moreover, hooks stop all counting and checks once the threshold is reached, and modern branch predictors generally handle these conditional checks efficiently.

For multi-threaded programs, the overhead can be more significant because counting and threshold checks require synchronization (e.g., atomic operations). Although hooks still cease checking after meeting the threshold, the overhead from the end marker's hook is unavoidable.

To mitigate this, we propose two strategies:

- Identifying a lower-overhead marker.

- Using the program counter to track the marker instead of hook instrumentation when running in simulation.

Lower Overhead Marker To reduce overhead on real hardware, we can trade some precision by selecting an IRBB near the end of the interval that executes less frequently than the final IRBB. Because hook overhead is tied to execution frequency, this choice reduces cost. We do this by defining a search distance in IR instructions from the interval's end, identifying candidate IRBBs within this distance using the count-stamp vector, and selecting the least frequently executed one using the IRBB vector. This effectively marks a point slightly before the interval's true end, reducing overhead at the cost of some precision.

Program Counter Tracking in Simulation While hook instrumentation is necessary on real hardware, when running *nuggets* in a simulator we can leverage simulator capabilities to eliminate hooks entirely. The LLVM nugget creation pass labels the marker IRBBs in the assembly, and we use the program counter addresses of these labels as inputs to the simulator to track markers. This completely removes marker overhead in simulation, ensuring measurements remain unaffected.

3.5 Sample Validating

Nugget addresses the limitations of using simulation to validate selected samples (Section 2.3) by generating *nuggets* that can run directly on both real hardware and simulation. Therefore, we can validate the selected samples in real hardware, and use them for simulation. This approach allows for fast, scalable validation of sampling accuracy without the high overhead of simulation, even for large applications.

Because nuggets are **binary-independent**, they can be validated across different systems with minimal constraints. Validating the same samples on multiple platforms enhances confidence in their *representativeness and robustness*. As shown later in Section 5.1, we found that having *consistent prediction error across platforms* is a stronger indicator of sample quality than achieving low error on a single platform.

To run a nugget on real hardware, the system must still execute the portion of the

program leading up to the sample's start marker (Section 3.4.1). For samples that are in later stages of a large program's execution, this can introduce some validation overhead.

In the future, we plan to reduce this overhead further by exploring techniques such as creating software checkpoints using CRIU [7] or BLCR [23] to resume execution closer to the sample start point.

Chapter 4

Evaluation

In this section, we revisit the goals of Nugget and evaluate how well we achieve them.

1. Analyzing program execution quickly and without restrictions on architecture or host machine.

2. Automatically generating samples that are independent of the binary.

3. Quickly validating the selected samples for the targeted benchmarks and input size.

We begin by evaluating the first goal through measurements of the analysis overhead introduced by Nugget, comparing it to both uninstrumented execution and to functional simulation (Section 4.1). We then address the second and third goal by employing two simple sample selection methodologies, random sampling and LLVM IR basic block (IRBB) vector-based k-means sampling, and evaluating their effectiveness (Section 4.2).

Model	Core	Cache Hierarchy	Memory
AMD Ryzen Threadripper 3960X	24-core OOO @ 3.8 GHz	L1d:32 KiB L1i:32 KiB L2:512 KiB L3:128 MiB	DDR4 2133 MHz 136.5 GiB/s
Ampere Altra	160-core OOO @ 2.8 GHz	L1d:64 KiB L1i:64 KiB L2:1 MiB SLC:64 MiB	DDR4 3200 MHz 409.6 GiB/s

Table 4.1: Real Hardware Configuration

Suite	Workload	Parallelism	Lang.
SPEC CPU2017	Desktop and server workloads	Single-threaded	C, C++, Fortran
NPB	HPC kernels	Multi-threaded (OMP)	Fortran, C
LSMS	First-principles materials simulation	Single-threaded	C++

Table 4.2: Overview of Benchmark Suites Used in Evaluation

Table 4.1 presents the detailed configurations of the real hardware used in our evaluation. For the simulation models, we employ a basic generic functional model using the ATOMIC CPU in gem5, running a guest Ubuntu 24.04 environment, to perform interval analysis on both x86 and Arm architectures.

We evaluate Nugget using the SPEC CPU 2017 benchmark suite, the NAS Parallel Benchmarks (NPB) suite [8], and the LSMS benchmark [46], covering a wide range of realistic workloads as summarized in Table 4.2.

We use the "-O2" optimization level for the compilation. The input chosen for each benchmark is too large to be simulated in feasible time (except for NPB class "A").

For SPEC CPU2017, we use the reference input sizes, which require between 198 billion and 9,897 billion instructions. For NPB, we use class "C" and "D" inputs, with class "C" requiring between 24 billion and 2,714 billion instructions and class "D" ranging from 1,217 billion to 37,533 billion instructions. For LSMS, we use the Fe input, which requires between 28,484 billion and 198,885 billion instructions (one binary per ISA).

We used SPEC CPU2017 Integer suite, except for "619.lbm_s" and "644.nab_s", which are from the Floating-Point suite. We do not include CPU 2017 benchmarks that failed to compile with the original SPEC CPU2017 GCC build system (6 benchmarks) or en-

countered Fortran linking issues (5 benchmarks). We did not encounter the same Fortran linking issues with any NPB Fortran benchmarks.

To accurately measure hardware performance, we employ *cpuset* to strictly bind workloads to specific cores and memory nodes. We also disable CPU frequency scaling and fix the CPU at a constant frequency. The LLVM passes are implemented on top of LLVM 19.0.0git.

4.1 Interval Analysis Efficiency (Goal 1)

In this section, we evaluate the efficiency of Nugget's interval analysis. We define efficiency using two criteria: (1) *low slowdown*, quantified as the ratio between the time spent on interval analysis and the time required for executing the full workload; and (2) *flexibility*, meaning the ability to perform the analysis across different host machines.

During interval analysis, we identify all valid intervals and collect two key artifacts: the LLVM IR Basic Block (IRBB) vector and the count-stamp vector (Section 3.3.2). These outputs are later used to generate nuggets.

For all workloads, we execute the complete preparation and analysis pipeline shown in Figure 3.1. Additionally, we compile an unmodified baseline binary from the same LLVM IR to isolate and measure the overhead introduced solely by the interval analysis.

We begin by comparing the overhead of interval analysis when performed via functional simulation versus Nugget. Next, we evaluate how overhead varies across a range of workload types and use different host machines to demonstrate the flexibility of Nugget. Finally, we study how overhead scales with the number of threads in the workload, noting that synchronization requirements during interval analysis can amplify slowdown at higher thread counts.

4.1.1 Comparison with Functional Simulation

For this experiment, we use the NPB suite with Class "A" inputs and 4 threads. We select Class "A" because larger input sizes, such as those used in Figure 4.2, would require months to analyze using functional simulation. We use multi-threaded workloads to demonstrate a worst-case overhead scenario, since interval analysis must track synchro-

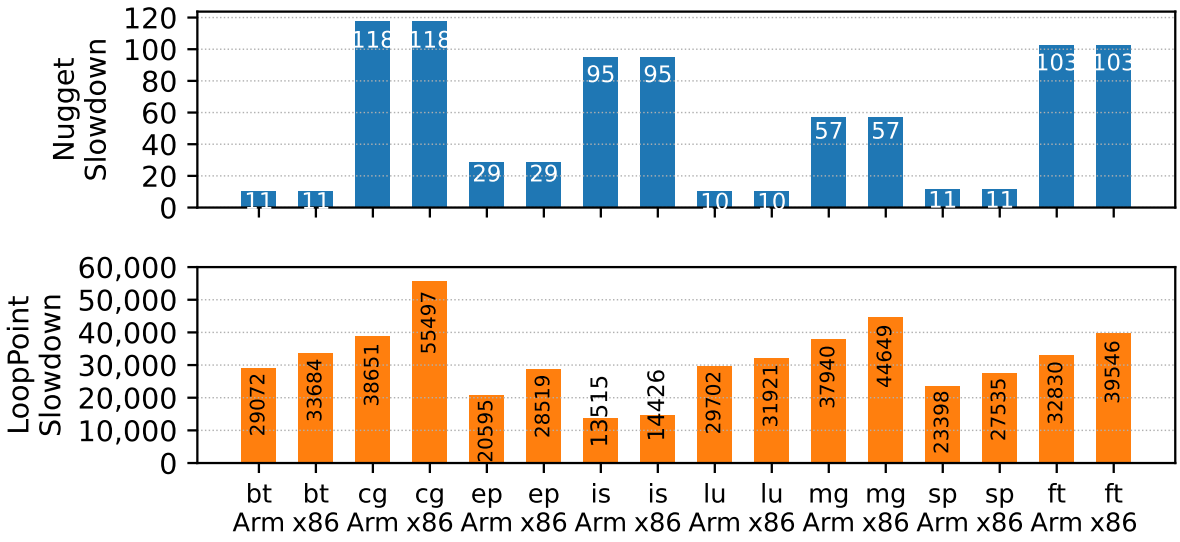

Figure 4.1: Interval analysis overhead via Nugget and functional simulation.

nization events between threads. Additionally, multi-threaded workloads reflect current trends in modern computing, making them a relevant evaluation target.

To evaluate the efficiency of Nugget, we compare it against LoopPoint [39], a recent state-of-the-art sampling methodology. LoopPoint is selected for comparison because it supports multi-threaded workload sampling and has integrated support within the gem5 simulator for full-system analysis [37]. This is particularly relevant, as all nuggets in our framework are generated on real hardware, a full-system environment.

As shown in Figure 4.1, we observe an average 577.65× reduction in interval analysis overhead when using Nugget instead of gem5's functional simulation. The overhead on the order of tens of thousands in functional simulation highlights its impracticality for interval analysis in large applications. On average, Nugget incurs a slowdown of 54×, with a maximum of 118× for the "cg" benchmark. By comparison, LoopPoint's average overhead is 31,343×, peaking at 55,497× for "cg-x86."

Overall, multi-threaded workloads amplify interval analysis overhead because their baseline execution times are significantly shorter (due to parallelism) while the analysis itself incurs greater cost from synchronization requirements. The single-threaded workloads in Figure 4.2 exhibit much lower overheads, which we discuss further in Section 4.1.2. The "cg" benchmark has the shortest execution time in this experiment (0.28 seconds),

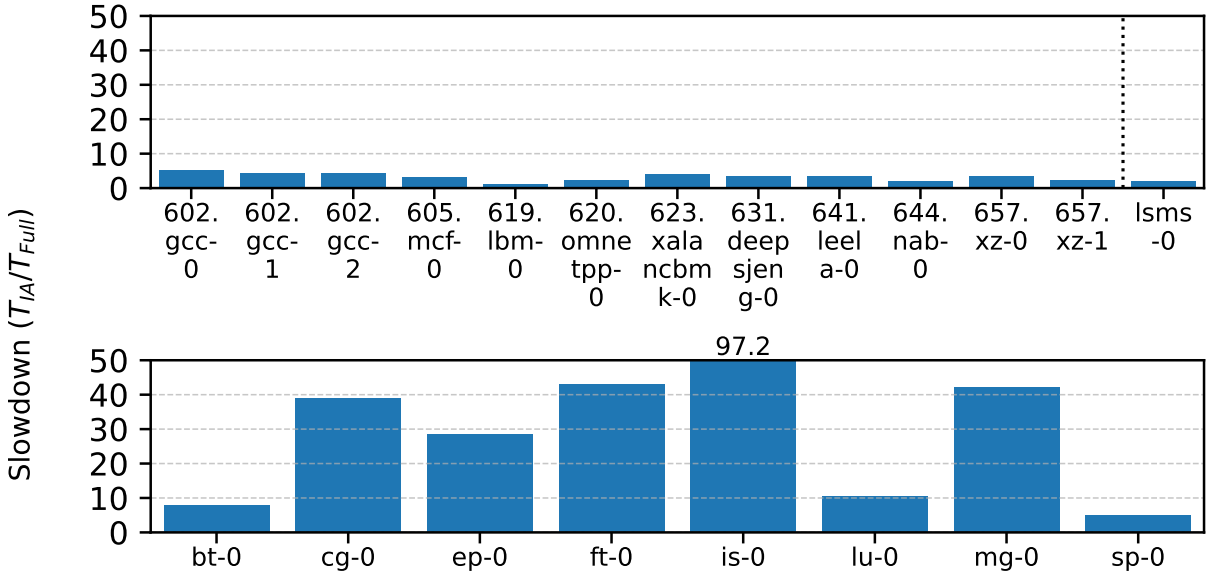

Figure 4.2: Interval analysis overhead via Nugget across different workloads.

which further amplifies the measured overhead for both methods. The "is" benchmark exhibits the lowest overhead under functional simulation but still incurs a 95× slowdown with Nugget, primarily due to the frequent executions of interval analysis hooks, as discussed in Section 4.2.3. The "mg" and "ft" benchmarks show relatively high overhead for both methods because their parallel regions increase synchronization costs during interval analysis, which in turn adds further simulation overhead. We examine the relationship between synchronization and overhead in more detail in Section 4.1.3.

In addition to its lower overhead, Nugget offers a key practical advantage: it requires only a single interval analysis run to generate nuggets that can be executed across multiple architectures. In contrast, LoopPoint must repeat interval analysis for every binary variant or target architecture, significantly increasing its total cost.

4.1.2 Mix of Workload Types

We also evaluate overhead across a mix of workload types. For single-threaded workloads, we use the SPEC CPU2017 suite with the `ref` input size and the LSMS benchmark with the `Fe` input size. For multi-threaded workloads, we use the NPB suite with Class "C" in this experiment.

Figure 4.2 shows the interval analysis slowdown across the evaluated workloads and

demonstrates that **the overhead varies significantly depending on the level of synchronization**. The average observed overhead is 3× for SPEC CPU2017, 2× for LSMS, and 34× for NPB. The overhead remains low for SPEC CPU2017 and LSMS because they are single-threaded workloads, which simplifies interval analysis by eliminating the need for synchronization. In contrast, the high overhead for NPB arises from the synchronization required during multi-threaded interval analysis, particularly due to aggregation across threads. As synchronization demands increase, so does the overhead, a relationship we explore further in Section 4.1.3.

To demonstrate the flexibility of our approach across host machines, we repeat the interval analysis on two distinct systems listed in Table 4.1 and find consistent results across both platforms for single-threaded workloads, with one corner case. For SPEC CPU2017, interval analysis results are identical across the two systems. For NPB, the multi-threaded nature of the benchmarks introduces variability between runs (even on the same host), a well-known challenge in multi-threaded sampling that is beyond the scope of this work [14, 39]. For LSMS, we observe minor differences in the execution frequency of some IRBBs, which we attribute to differences in floating-point precision across CPUs. In particular, the number of iterations in certain data-driven loops in LSMS is influenced by the underlying architecture's precision. Such differences could affect nugget representativeness if a marker IRBB is sensitive to these variations. Nevertheless, as shown in Section 4.2.2, nuggets generated from interval analysis on one system still achieve low prediction error when validated on the other system for LSMS.

4.1.3 Scaling with Number of Threads

Finally, we study how interval analysis overhead scales with the number of threads. For this experiment, we use the NPB suite with Class "D" inputs and vary the thread count across {1, 4, 8, 16}. To ensure fair comparisons, all overheads are normalized to the execution time of the original workload running with a single thread.

As shown in Figure 4.3, most workloads exhibit increasing overhead with higher thread counts, consistent with our expectation that more threads lead to greater synchronization overhead. However, this growth is not uniform across all workloads because not all of

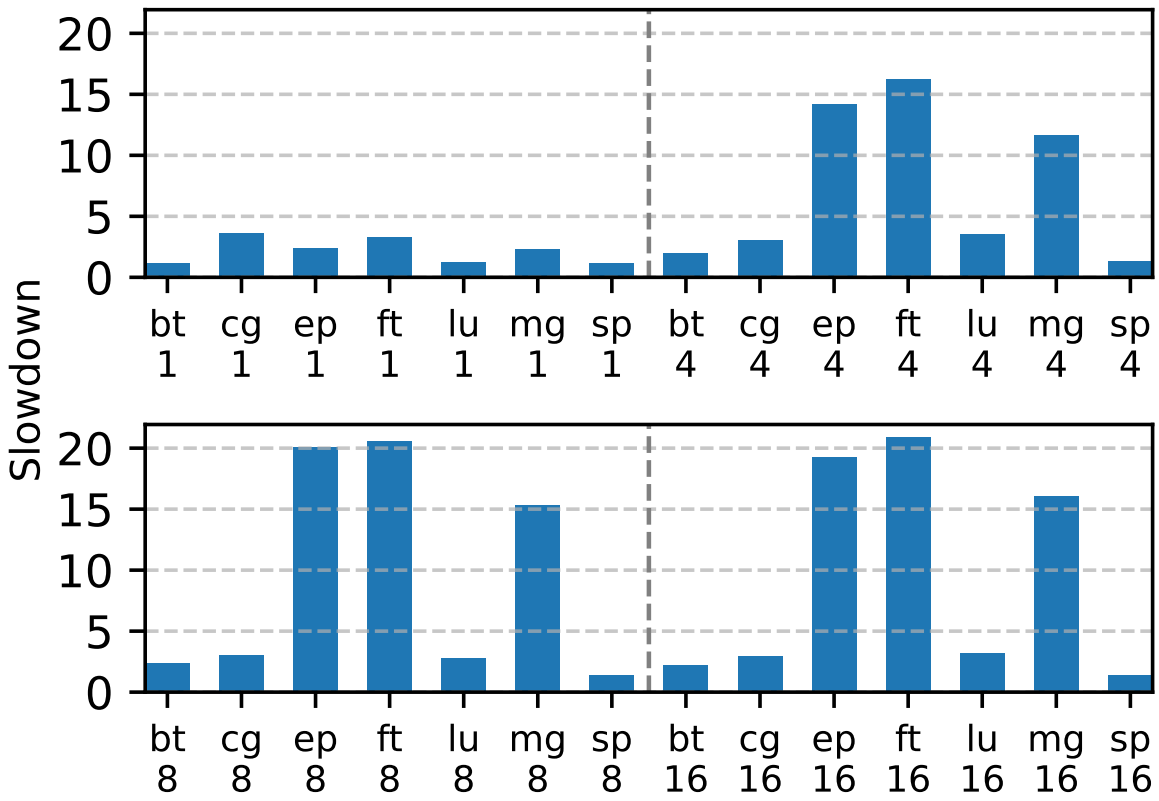

Figure 4.3: Interval analysis overhead via Nugget across different numbers of threads.

them require the same level of synchronization during interval analysis, as not all hooks are executed within parallel regions. For "bt", "cg", "lu", and "sp", the consistently low slowdown as the thread count increases suggests that hooks are not frequently executed within parallel regions for these benchmarks. In contrast, "mg" and "ft" show a significant increase in overhead with more threads, indicating that these benchmarks require more synchronization during interval analysis—likely due to the complexity of their parallel regions.

Even at higher thread counts (up to 16×), Nugget maintains overheads that are orders of magnitude lower than those of functional simulation-based interval analysis (Figure 4.1).

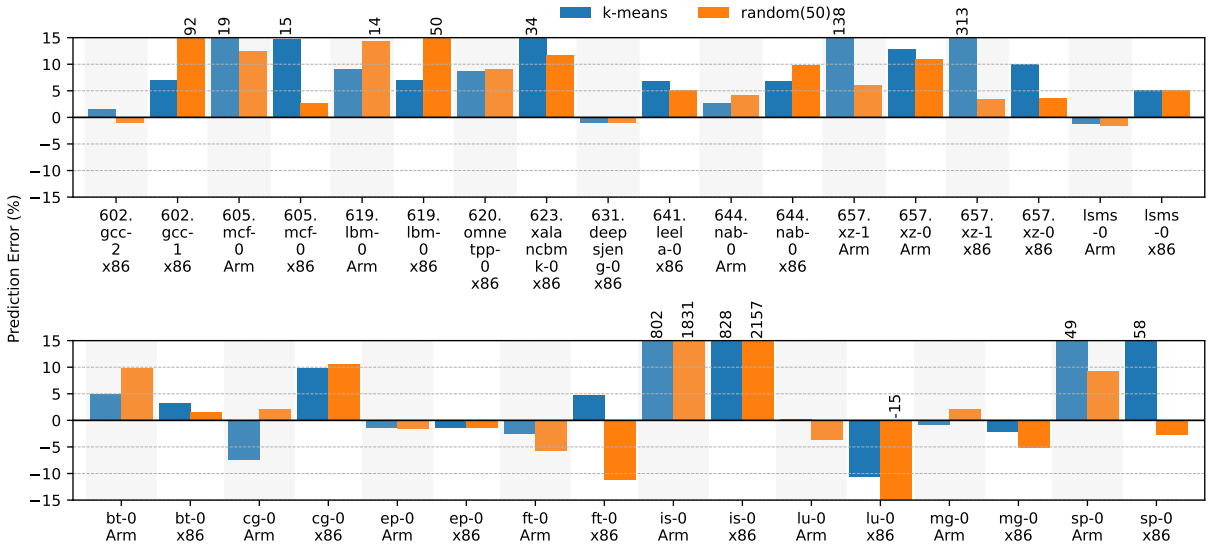

Figure 4.4: Prediction error by machine and sampling method. **All results shown are obtained from executing the nuggets directly on real hardware**, without using simulation.

4.1.4 Summary

Our experiments show that Nugget achieves orders-of-magnitude lower interval analysis overhead compared to functional simulation, with an average reduction of over 577.65× relative to LoopPoint for multi-threaded workloads. It also produces consistent analysis results across different host machines, demonstrating its portability.

By enabling fast, binary-independent interval analysis, even for large workloads, Nugget successfully meets Goal 1.

In the following sections, we build upon this foundation to demonstrate how Nugget supports a variety of sampling methodologies (Goal 2) and enables rapid validation of selected samples on real hardware (Goal 3), further establishing its effectiveness for sampling methodology research.

4.2 Applying Sampling Methodologies to the Nugget Framework and Validating the Selected Samples (Goals 2 & 3)

The primary objective of Nugget is to support diverse sample selection methodologies while providing efficient interval analysis, binary-independent samples, and scalable validation capabilities. In this section, we leverage these features to execute the selected samples, *nuggets*, directly on real hardware, enabling us to assess whether different sampling strategies can accurately predict full workload performance *without relying on simulation*.

Because this paper does not propose new sampling algorithms, we focus on two straightforward approaches: random sampling and LLVM IRBB vector-based k-means clustering. We show that Nugget seamlessly implements these methodologies and facilitates validating the selected samples on *real hardware*. We also show that different workloads may benefit from different sampling strategies; Nugget enables discovering and applying methods tailored to specific workload characteristics. Moreover, we highlight the importance of validating sampling methodologies across multiple machines, especially when the target system is not yet available. Finally, we discuss the overhead introduced by the hooks used to mark sample intervals within nuggets.

4.2.1 Sampling Methodologies

For the random sampling approach, we randomly select a set of intervals from the program execution to estimate its average or cumulative properties. This selection method is based on statistical sampling methods [49].

For the clustering approach, we use the k-means algorithm to cluster the IRBB vectors, select "k" samples to represent the program phases of the workload, and use the cluster weights to estimate the average or cumulative properties of the program execution. This approach is based on basic block vector (BBV) phase-identification techniques [16] (e.g., SimPoint) and on the fact that the IRBB vector is an abstraction of the actual BBV.

We refer to the random approach as "Random" and the clustering approach as "K-means". We use 50 samples as the target number for both methodologies, which is

statistically large enough to approximate the population distribution [25]. We apply the Silhouette score with the K-means algorithm to determine the optimal number of clusters for each workload, enforcing the constraint #clusters ≤ 50 (with $k = 50$ as the upper bound). We assign equal weights to all intervals in the "Random" approach, whereas the "K-means" approach weighs each interval according to the size of its cluster.

4.2.2 Sample Validation

We use the analysis information collected in Section 4.1.2 to select samples with the "Random" and "K-means" approaches. We then generate nuggets for the selected samples and validate them as described in the Sample Creation and Validation step in Figure 3.1. The nuggets are used to predict the runtime of each workload, with the runtime of the full execution serving as the baseline for calculating prediction error. We choose runtime as the metric because it is a common target of sampling methodologies [39], and IPC has been shown to be unsuitable for multi-threaded workloads [2].

Figure 4.4 presents the prediction errors of nuggets generated by the Random and K-means approaches for each workload on the real-hardware machines listed in Table 4.1. As shown, many workloads exhibit significant errors, often exceeding the accuracy reported by prior methodologies such as SimPoint (3% for single-threaded workloads) and LoopPoint (2.9% for 8-threaded workloads).

Several factors contribute to these elevated errors. (1) The sampling methodologies are naively designed. For example, the Random approach relies heavily on luck and the specific composition of the interval pool. (2) Previous work rarely validates samples on real hardware [14, 22, 26, 30, 32, 36, 39–41, 49, 50]. Instead, they mostly rely on user-mode-only simulation, which is typically less noisy due to more controllable thread interleaving compared to actual hardware [1, 3, 9, 17]. In Section 5.1, we further analyze the sources of prediction error using smaller workloads in simulation. (3) Prior work rarely validates large workloads as we used in our experiments (e.g., reference input sizes for CPU2017). Larger and more realistic applications exhibit increasingly complex behaviors [26, 31, 39, 41, 48, 51], which demand more representative sample sets, as discussed in Section 2.3. In such cases, large program phases can amplify prediction error. (4) Hardware-based

nuggets incur additional overhead for the begin and end interval markers, particularly in multi-threaded contexts, which further inflates prediction error. We provide a detailed discussion of this overhead in Section 4.2.3.

As the figure shows, the sampling approach that yields the lowest prediction error varies by workload. For example, for "sp," the "Random" approach with 50 samples achieves the lowest prediction error across all methods, and this holds on both machines, suggesting that this sample set is architecturally and microarchitecturally independent. In contrast, for "lbm," the "K-means" approach consistently yields the lowest prediction error.

We focus on demonstrating Nugget's ability to validate samples on distinct machines in this section. In Section 5.1, we will further analyze how to utilize the hardware validation data with a case study. We also find that maintaining consistency across machines is a stronger indicator of sample quality than minimizing prediction error on any single platform. This insight further underscores the importance of efficient, cross-platform validation in the design of robust sampling methodologies.

4.2.3 Hook Overhead

Nugget uses hook instructions to mark sample intervals, which can introduce runtime overhead during workload execution. In Figure 4.5, we present the number of times the marker hooks are executed for the randomly selected nuggets shown in Figure 4.4. The number of hook executions is normalized to the total number of IRBB executions during all nugget executions for each workload and input pair. As shown, "is" exhibits a significantly higher number of hook executions compared to other workloads, indicating that the prediction error observed for "is" in Figure 4.4 is influenced by the excessive overhead from hook executions. Because "is" frequently executes only a small number of IRBBs within a parallel region, it triggers an excessive number of hook invocations. "is" was also one of the workloads with the largest overheads for analysis as shown in Figure 4.1. This leads to substantial overhead and results in inaccurate prediction errors. Hook overhead can only slow execution, so it can cause false negatives (rejecting good samples) but is unlikely to cause false-positive conclusions.

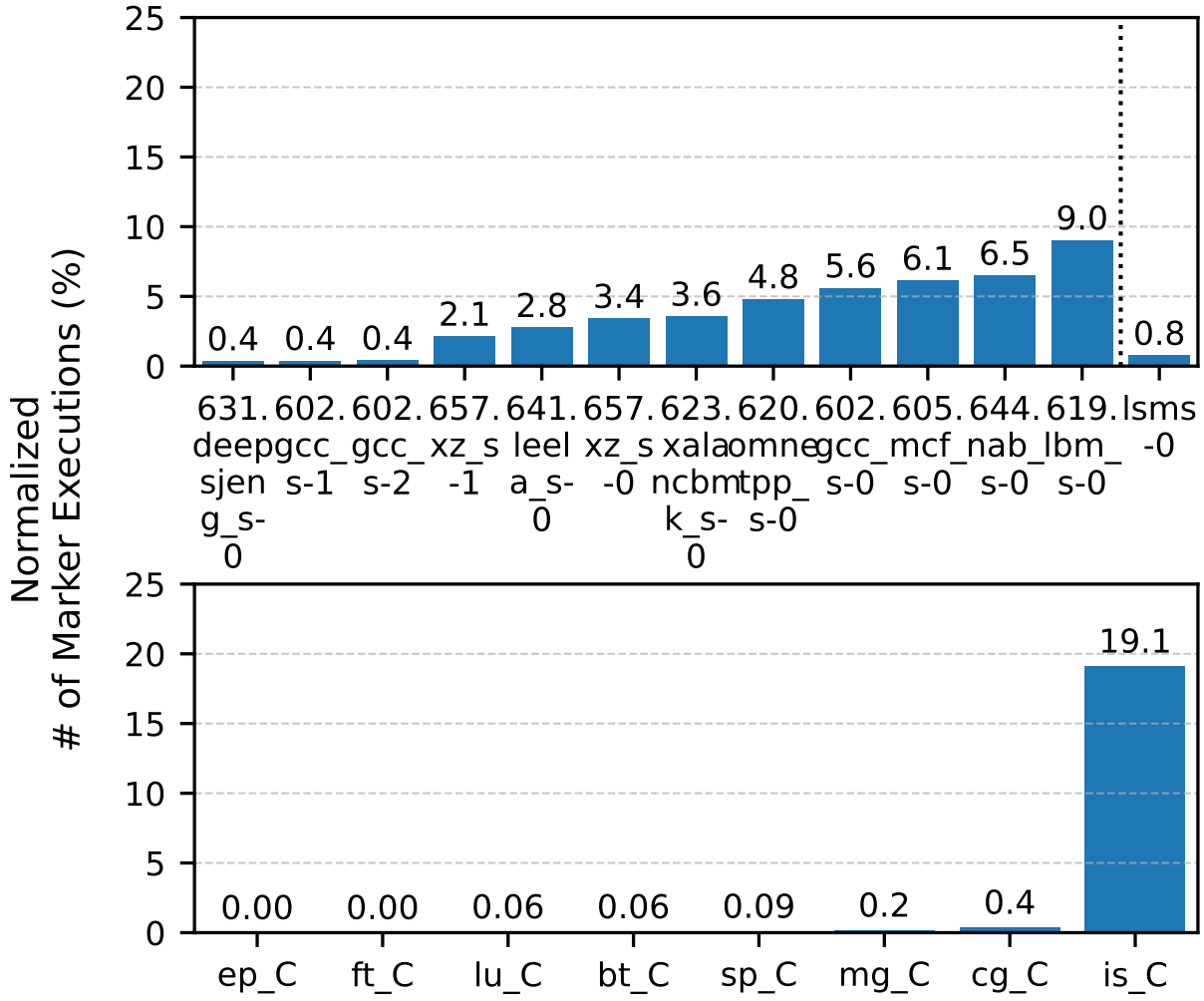

Figure 4.5: The number of marker hook executions for the randomly selected nuggets shown in Figure 4.4. The number of hook executions is normalized to the total number of IRBB executions during all nugget executions for each workload and input pair.

For other workloads, the number of hook executions does not show a clear correlation with the prediction error observed in Figure 4.4. Cleanly separating the impact of hook overhead from other sources of prediction error is still challenging. The number of hook executions and the prediction errors observed suggest that the cutoff line for a good marker should be executing lower than 10% for single-threaded workloads and lower than 1% for multi-threaded workloads, in terms of the total IRBBs executed in the nugget, to avoid significant overhead impacting the validation results. Multi-threaded workloads are more sensitive to hook overhead due to the synchronization required for counting and threshold checks, as discussed in Section 3.4.2.

This behavior highlights a limitation of the current hook-based interval marking approach. Future work could explore alternative mechanisms to reduce this overhead or to decouple it from the validation results. However, addressing this issue is nontrivial, especially for multi-threaded workloads: overhead in parallel regions can differ significantly from that in serial regions, and parallelism may sometimes mask or amplify hook-related costs, complicating the isolation of their impact.

It is important to note that this overhead is unique to real hardware execution. In simulation, as discussed in Section 3.4.2, sample intervals can be marked directly by the simulator without introducing runtime overhead.

Chapter 5

Case Studies

In this section, we present two case studies. First, we separate the overhead of nugget markers from the accuracy of the sampling selection process by running the nuggets both on hardware and in simulation. Then, we illustrate how nuggets can serve as lightweight microbenchmarks to assess simulation model accuracy, eliminating the need to develop dedicated microbenchmark suites.

5.1 Nuggets in Simulation and on Hardware

The goal of computer architecture analysis is often to compare the performance of two systems. Therefore, to investigate the usefulness of nuggets, we run both the entire workload and the nuggets and compare the speedup between different systems. We compare four different systems *in simulation* to eliminate the potential inaccuracies from the interval selection marker in hardware. We also compare four different hardware systems. With these eight systems, we can make two conclusions. (1) We can use hardware to provide increased confidence in the selected intervals, but we cannot prove or bound the error due to the overheads discussed in Section 5.1.2. (2) We find that Random and basic-block vector approaches to interval selection are not microarchitectural independent. In fact, we find that microarchitectural differences affect the accuracy of the selection methodology more than the ISA.

In this study we report the *error in predicted speedup*, not the error in absolute runtime. We found that in cases with modest absolute error the predicted speedup (or relative

Model	ISA	Core	Cache Hierarchy	Memory
Config 1	x86	4-core Large OOO @ 4 GHz	L1d: 64 KiB L1i: 64 KiB L2: 1 MiB	DDR4 2400 MHz 38.4 GB/s
Config 2	Arm	8-core Small OOO N1 @ 3 GHz	L1d: 64 KiB L1i: 64 KiB L2: 1 MiB SLC: 32 MiB	DDR4 2400 MHz 76.8 GiB/s
Config 3	x86	4-core Base OOO @ 4 GHz	L1d: 64 KiB L1i: 64 KiB L2: 1 MiB	DDR4 2400 MHz 38.4 GB/s
Config 4	Arm	8-core Base OOO @ 4 GHz	L1d: 64 KiB L1i: 64 KiB L2: 1 MiB	DDR4 2400 MHz 76.8 GiB/s

Table 5.1: gem5 Simulation Model Configuration. The large, small, and base out-of-order cores are internal models that target different performance characteristics.

performance) is often much more accurate.

5.1.1 Experimental Setup

We use the selected nuggets to conduct experiments on both gem5, using full-system simulation, and real hardware. In addition to the systems listed in Table 4.1, we include two additional real hardware platforms: an Intel i9-9900X system and an Intel i7-9700 system. The simulation models used are listed in Table 5.1 and run a guest Ubuntu 24.04 kernel, matching the software environment of the real hardware.

Sample sets are selected using the "Random" and "K-means" methods described in Section 4.2.1. We set $k = 20$ in this experiment, as the use of NPB Class A with a single-threaded configuration results in a relatively small pool of intervals available for sampling. Single-threaded workloads also help eliminate confounding factors such as thread interleaving, which could otherwise affect our measurement of prediction error.

In simulation, we use gem5 to track sample interval markers directly, thereby avoiding the runtime overhead introduced by hook-based marking mechanisms. To simulate nuggets efficiently, we first use KVM to fast-forward execution to the warmup marker and then create a gem5 checkpoint [21]. This allows us to resume nugget execution from that point using different microarchitectural configurations, subject to certain constraints (e.g., the number of cores must remain fixed). For each nugget, we restore from the checkpoint and begin measurement only after reaching the start marker, allowing the microarchitectural state to warm up naturally.

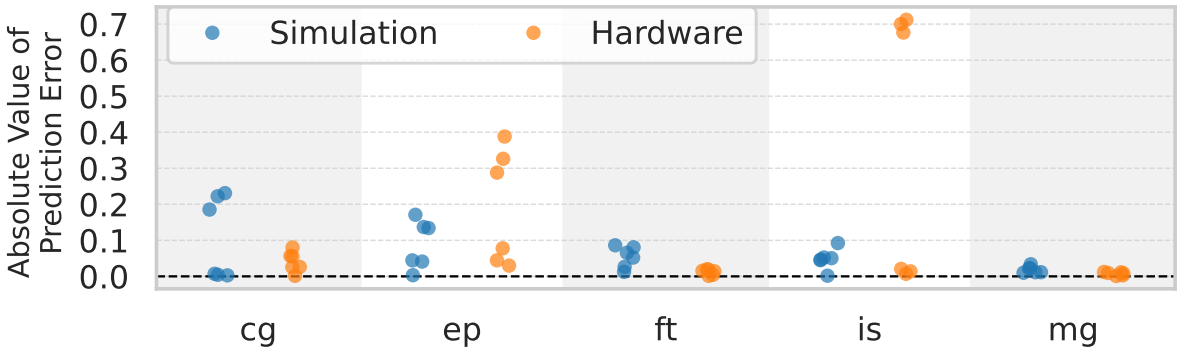

Figure 5.1: Distribution of absolute speedup prediction errors using K-means samples across simulation (blue) and hardware (orange) comparisons for selected benchmarks. "0.1" means 10% difference in predicted speedup.

5.1.2 Results

Hardware-Based Validation Offers Scalability and Early Signal Validating samples on real hardware is significantly faster and more scalable than using simulation. Several simulation runs (e.g., "bt", "lu", and "sp"), even with small inputs such as Class A, remain incomplete after two weeks, making simulation impractical for extensive validation. By contrast, hardware validation enables us to quickly assess the variability in speedup prediction across platforms.

Figure 5.1 shows the distribution of absolute speedup prediction errors for selected benchmarks. Each point is absolute value of the error of one pair of configurations' (in simulation) or pair of hardware systems' speedup compared to the true speedup of those two systems determined by running the entire workload. The hardware results allow us to quickly identify benchmarks with high variability in prediction error, such as "cg", "ep", and "is", which exhibit wide error ranges across different platforms. For these workloads with large variations or large errors on hardware, we find the intervals are *not representative of the true speedup*. However, we cannot use the error on hardware to bound the error in simulation (e.g., the error for ft in simulation is larger than on hardware). The main reason we cannot use the hardware error to bound the simulation error is that the error is strongly affected by the underlying microarchitecture. In the next section, we analyze the sources of prediction error in more detail using simulation data and controlled

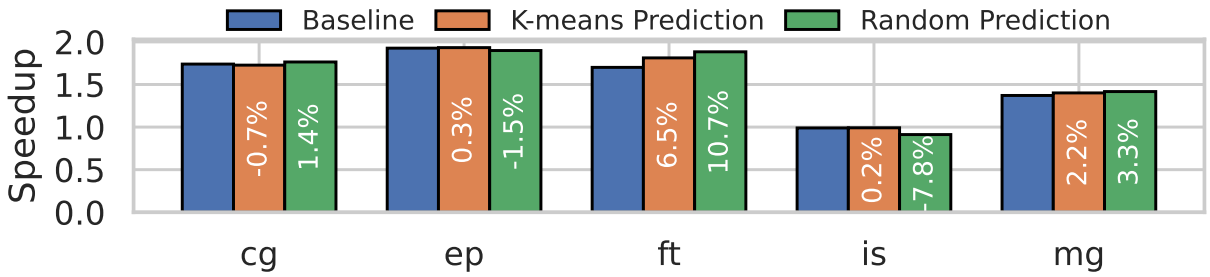

Figure 5.2: Speedup: Random vs K-means samples when ISA varies and microarchitecture is fixed, Config 3 vs Config 4.

case studies.

ISA vs. Microarchitecture: Case Study To isolate the sources of prediction error, we compare three types of platform pairs: (1) systems with different ISAs but small microarchitecture differences, (2) systems with significantly different microarchitectural configurations, but the same ISA, and (3) systems that differ in both ISA and microarchitecture.

Figure 5.2 shows the speedup prediction error when the main difference between platforms is the ISA (Config 3 vs Config 4). In this experiment, the ISA has a noticeable impact on the binaries: for instance, the Arm binaries have vectorization enabled, whereas the x86 binary does not. The average absolute speedup prediction error is 5% for Random-selected nuggets and 2.0% for K-means-selected nuggets. Among all benchmarks, "ft" exhibits the highest K-means error (6.5%), likely due to an unrepresentative sample set. A linear regression of nugget runtimes across the two ISAs explains 94.5% of the variance, indicating that the selected samples capture similar execution phases across architectures. The K-means speedup error for "ft" is consistent with its runtime prediction errors on Config 3 (4.9%) and Config 4 (–1.5%), suggesting the overestimation stems from sample imbalance. This implies that certain performance-critical phases were not captured by the selected nuggets. The Random-selected samples show a similar pattern for both the "ft" and "is".

Figure 5.3 shows the speedup prediction error when the microarchitecture varies: Config 4 over Config 2 for Arm and Config 3 over Config 1 for x86. The average absolute prediction error increases to 8.1% for Random-selected nuggets and 5.9% for K-means-

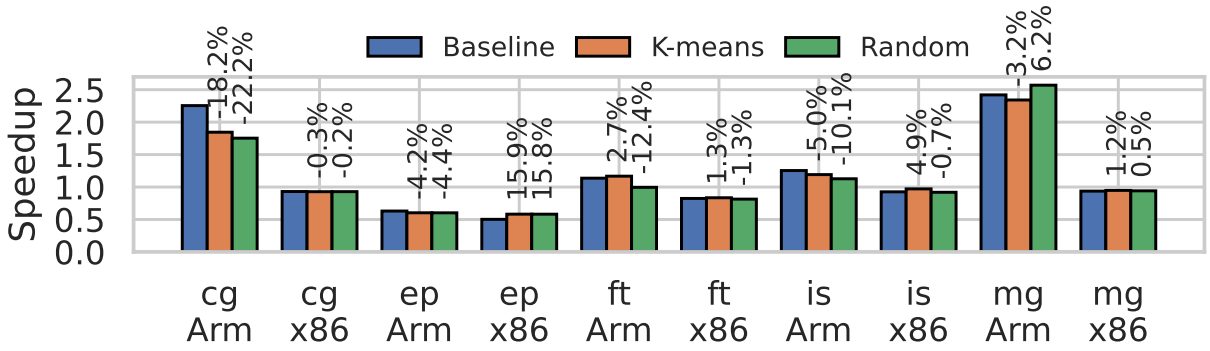

Figure 5.3: Speedup between Random and K-means samples when microarchitecture varies and ISA is fixed: Config 4 over Config 2 for Arm, and Config 3 over Config 1 for x86.

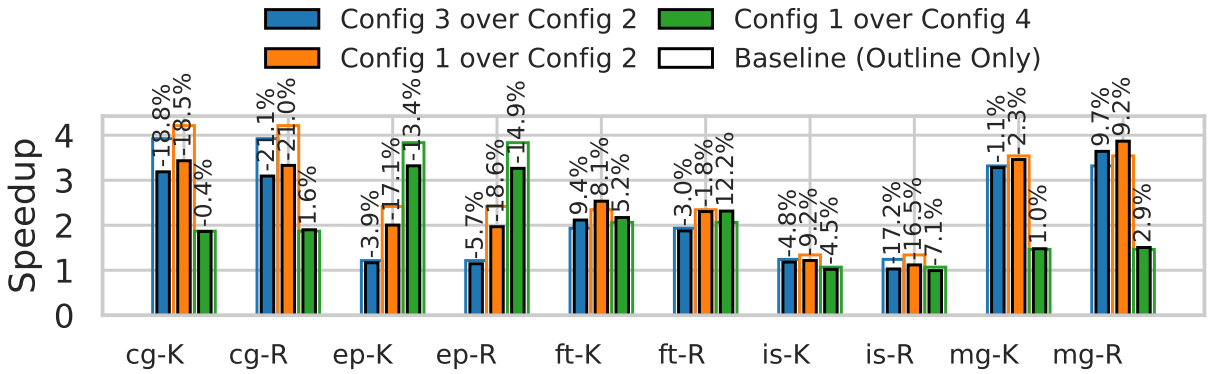

Figure 5.4: Speedup comparison between Random and K-means samples when both ISA and microarchitecture vary across simulation models. "K" indicates K-means samples, and "R" indicates Random samples.

selected nuggets. The comparison between Config 4 and Config 2 for Arm results in higher error in predicted speedup using the selected nuggets than between Config 3 and Config 1 for x86, reflecting greater microarchitectural differences in the Arm systems despite running the same benchmarks and sample sets.

Figure 5.4 presents the speedup prediction error when both ISA and microarchitecture vary across simulation models. The average absolute error rises to 10.8% for Random-selected nuggets and 7.9% for K-means-selected nuggets. The lowest errors occur in comparisons between Config 1 and Config 4 (7.7% for Random, 5% for K-means), where microarchitectural differences are relatively minor. In contrast, larger errors are observed in comparisons involving Config 2, which diverges significantly from other configurations

in both ISA and microarchitecture. These findings highlight the sensitivity of prediction accuracy to microarchitectural differences more than to ISA variation.

In summary, while this paper does not propose a new method for selecting representative intervals, we show the importance of comparing different interval selection techniques across a wide variety of architectures and microarchitectures. Nugget enables future work to have apples-to-apples comparisons across architectures and fast comparisons on hardware opening this new line of research.

5.2 Model Accuracy Evaluation

In this section, we present how we used nuggets to identify sources of errors in gem5's models. Identifying sources of errors in the simulation model is key to establishing baseline performance for evaluating architectural ideas.

Microbenchmarks are a common tool for fine-tuning architectural simulators. To build a useful model of the Ampere Altra processor, we developed a collection of models including CPU, cache, and memory models using publicly available information on the hardware [6]. We used a set of microbenchmarks and fine-tuned the configuration of the models to lower the error in simulated cycles [45]. While we were able to reduce cycle count error to below 5% for many of the microbenchmarks, the models struggled to accurately simulate real applications such as NAS Parallel Benchmarks—with errors exceeding 600% [8].

Using microbenchmarks has two main limitations. (1) In many cases the compiler optimizations can remove essential parts of the code, resulting in false positives. (2) Even when preserved after compiler optimizations, these microbenchmarks often exercise a narrow slice of the design space, missing the complex interactions between microarchitectural components present in real programs.

While it is possible to develop more sophisticated microbenchmarks, this approach is labor-intensive and does not scale well as architectures develop. Nuggets are an organic alternative to microbenchmarks; they capture the diverse instruction mix and control flow found in the real workloads and they are small enough to provide a focused window for

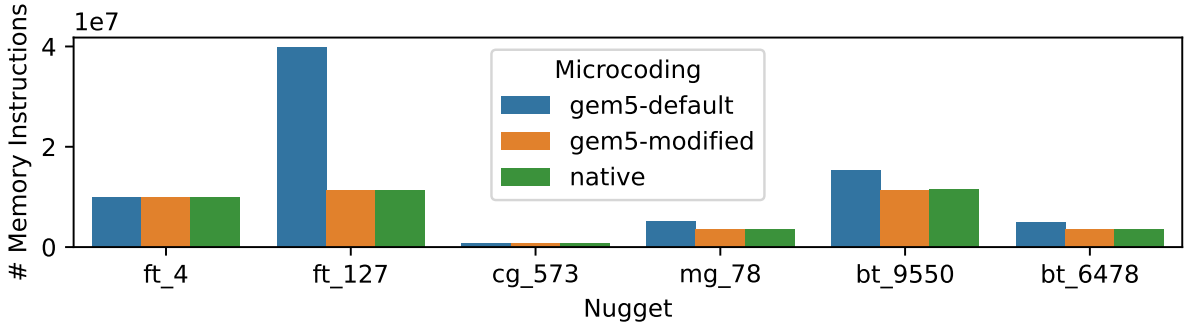

Figure 5.5: Comparison of the number of memory instructions for two different microcodings in gem5 vs real hardware.

manual inspection by the users.

To improve our model's accuracy, we collected hardware performance counters for 234 nuggets and compared the hardware counters to gem5 statistics for each nugget to identify the nuggets with the largest error. We found the data cache statistics explained most of the runtime error and for more than 97% of the nuggets, the error in the number of accesses to the data cache could be explained by differences in the number of memory instructions. We isolated the problem by inspecting execution traces from a small set of the nuggets. We observed that the nugget with the largest error was dominated by Arm's paired memory instructions (e.g., ldp, stp), while fewer were present in samples with small error.

Figure 5.5 shows a comparison of the number of memory instructions in our selected nuggets for three different microcodings. The orange bars show the number of memory instructions with the default microcoding of gem5; the blue bars show the number of memory instructions if the paired memory instructions were microcoded as one microop; while the green bars show the number of memory instructions measured on native hardware. The blue bars closely track the green bars, suggesting that the microcoding of paired memory instructions in gem5 is different from the native hardware.

Chapter 6

Conclusion

In this paper we presented Nugget: a framework for building program snippets that can be used for evaluating architectural ideas using full-system simulators. Nugget enables automatic instrumentation of a workload's binary for analysis, and for measurement. Binaries used for analysis can be run on real hardware much faster than prior work while the binaries used for measurement can be run on real hardware and simulators alike. Lastly, Nugget allows users to leverage real hardware to evaluate how well the selected samples represent the original workload.

Nugget is positioned to allow researchers to simulate workloads larger than the traditional sizes, enabling researchers to gain insights that were not possible before.

References

[1] A. Akram and L. Sawalha, "A Survey of Computer Architecture Simulation Techniques and Tools," *IEEE Access*, vol. 7, pp. 78 120–78 145, Jun. 2019.

[2] A. R. Alameldeen and D. A. Wood, "IPC Considered Harmful for Multiprocessor Workloads," *IEEE Micro*, vol. 26, no. 4, pp. 8–17, Jul. 2006.

[3] A. R. Alameldeen and D. A. Wood, "Variability in Architectural Simulations of Multi-Threaded Workloads," in *Proceedings of the 9th International Symposium on High-Performance Computer Architecture*, ser. HPCA '03. USA: IEEE Computer Society, Feb. 2003, pp. 7–18.

[4] F. Alexander, A. Almgren, J. Bell, A. Bhattacharjee, J. Chen, P. Colella, D. Daniel, J. DeSlippe, L. Diachin, E. Draeger, A. Dubey, T. Dunning, T. Evans, I. Foster, M. Francois, T. Germann, M. Gordon, S. Habib, M. Halappanavar, S. Hamilton, W. Hart, Z. (Henry) Huang, A. Hungerford, D. Kasen, P. R. C. Kent, T. Kolev, D. B. Kothe, A. Kronfeld, Y. Luo, P. Mackenzie, D. McCallen, B. Messer, S. Mniszewski, C. Oehmen, A. Perazzo, D. Perez, D. Richards, W. J. Rider, R. Rieben, K. Roche, A. Siegel, M. Sprague, C. Steefel, R. Stevens, M. Syamlal, M. Taylor, J. Turner, J.-L. Vay, A. F. Voter, T. L. Windus, and K. Yelick, "Exascale applications: skin in the game," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 378, no. 2166, p. 20190056, Mar. 2020.

[5] M. Alian, "Sustainable gem5 Simulations," https://www.gem5.org/assets/files/workshop-isca-2023/slides/sustainable-gem5-simulations.pdf, presented at the gem5 Workshop, ISCA 2023, Accessed: 2023-10-01.

[6] Ampere Computing. Ampere Altra Family Product Brief. [Online]. Available: https://amperecomputing.com/briefs/ampere-altra-family-product-brief

[7] F. Andrijauskas, I. Sfiligoi, D. Davila, A. Arora, J. Guiang, B. Bockelman, G. Thain, and F. Wurthwein, "CRIU – Checkpoint Restore in Userspace for computational simulations and scientific applications," Feb. 2024. [Online]. Available: https://arxiv.org/abs/2402.05244

[8] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrishnan, and S. K. Weeratunga, "The NAS Parallel Benchmarks," *Int. J. High Perform. Comput. Appl.*, vol. 5, no. 3, pp. 63–73, Sep. 1991.

[9] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, Aug. 2011.

[10] D. L. Bruening and S. Amarasinghe, "Efficient, Transparent, and Comprehensive Runtime Code Manipulation," Ph.D. dissertation, Massachusetts Institute of Technology, USA, 2004, aAI0807735.

[11] J. Bucek, K.-D. Lange, and J. v. Kistowski, "SPEC CPU2017: Next-Generation Compute Benchmark," in *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '18. New York, NY, USA: Association for Computing Machinery, Apr. 2018, pp. 41–42.

[12] S. Caculo, M. Madhav, and J. Baxter, "Memory Access Vectors: Improving Sampling Fidelity for CPU Performance Simulations," Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.02344

[13] T. E. Carlson, W. Heirman, and L. Eeckhout, "Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulation," in *SC '11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2011, pp. 1–12.

[14] T. E. Carlson, W. Heirman, K. Van Craeynest, and L. Eeckhout, "BarrierPoint: Sampled simulation of multi-threaded applications," in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Mar. 2014, pp. 2–12.

[15] E. Colp, J. N. Amaral, C. Benedicto, R. E. Rodrigues, and E. Borin, "Report: The 523.xalancbmk_r Benchmark (June 2016 Development Kit)," Department of Computing Science, University of Alberta, Technical Report, Jan. 2018, published January 18, 2018. Part of The Alberta Workloads for the SPEC CPU2017 Benchmark Suite.. [Online]. Available: https://webdocs.cs.ualberta.ca/~amaral/AlbertaWorkloadsForSPECCPU2017/reports/xalancbmk_report.html

[16] K. Criswell and T. Adegbija, "A Survey of Phase Classification Techniques for Characterizing Variable Application Behavior," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 224–236, Jan. 2020.

[17] J. Devietti, B. Lucia, L. Ceze, and M. Oskin, "DMP: deterministic shared memory multiprocessing," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XIV. New York, NY, USA: Association for Computing Machinery, Mar. 2009, pp. 85–96.

[18] J. Dongarra and P. Luszczek, *Livermore Loops*. Boston, MA: Springer US, 2011, pp. 1041–1043.

[19] L. Eeckhout, J. Sampson, and B. Calder, "Exploiting program microarchitecture independent characteristics and phase behavior for reduced benchmark suite simulation," in *IEEE International. 2005 Proceedings of the IEEE Workload Characterization Symposium, 2005.*, Oct. 2005, pp. 2–12.

[20] L. Eeckhout, *Computer Architecture Performance Evaluation Methods*, 1st ed. Morgan & Claypool Publishers, 2010.

[21] gem5 Project. (2025) Checkpoints — gem5 Documentation. [Online]. Available: https://www.gem5.org/documentation/general_docs/checkpoints/

[22] T. Grass, A. Rico, M. Casas, M. Moreto, and E. Ayguad, "TaskPoint: Sampled simulation of task-based programs," in *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2016, pp. 296–306.

[23] P. Hargrove and J. Duell, "Berkeley Lab Checkpoint/Restart (BLCR) for Linux Clusters," *Journal of Physics: Conference Series*, vol. 46, p. 494, Sep. 2006.

[24] J. W. Haskins and K. Skadron, "Memory reference reuse latency: Accelerated warmup for sampled microarchitecture simulation," in *2003 IEEE International Symposium on Performance Analysis of Systems and Software. ISPASS 2003.*, Mar. 2003, pp. 195–203.

[25] R. V. Hogg, E. A. Tanis, and D. L. Zimmerman, *Probability and Statistical Inference*, 10th ed. Boston: Pearson, 2020.

[26] T. Huffmire and T. Sherwood, "Wavelet-based phase classification," in *2006 International Conference on Parallel Architectures and Compilation Techniques (PACT)*, Sep. 2006, pp. 95–104.

[27] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, D. Lee, N. Pemberton, E. Amaro, C. Schmidt, A. Chopra, Q. Huang, K. Kovacs, B. Nikolic, R. Katz, J. Bachrach, and K. Asanović, "FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud," in *Proceedings of the 45th Annual International Symposium on Computer Architecture*, ser. ISCA '18. Piscataway, NJ, USA: IEEE Press, Jun. 2018, pp. 29–42.

[28] S. V. Kodakara, J. Kim, D. J. Lilja, W.-C. Hsu, and P.-C. Yew, "Analysis of Statistical Sampling in Microarchitecture Simulation: Metric, Methodology and Program Characterization," in *2007 IEEE 10th International Symposium on Workload Characterization*, Sep. 2007, pp. 139–148.

[29] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation," in *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, ser. CGO '04. USA: IEEE Computer Society, Mar. 2004, p. 75.

[30] P. Lavin, J. Young, and R. Vuduc, "Multifidelity Memory System Simulation in SST," in *Proceedings of the International Symposium on Memory Systems*, ser. MEMSYS '23. New York, NY, USA: Association for Computing Machinery, Oct. 2024.

[31] A. Limaye and T. Adegbija, "A Workload Characterization of the SPEC CPU2017 Benchmark Suite," in *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2018, pp. 149–158.

[32] C. Liu, A. Sabu, A. Chaudhari, Q. Kang, and T. E. Carlson, "Pac-Sim: Simulation of Multi-threaded Workloads using Intelligent, Live Sampling," Oct. 2023. [Online]. Available: https://arxiv.org/abs/2310.17089

[33] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, S. Bharadwaj, G. Black, G. Bloom, B. R. Bruce, D. R. Carvalho, J. Castrill'on, L. Chen, N. Derumigny, S. Diestelhorst, W. Elsasser, M. Fariborz, A. F. Farahani, P. Fotouhi, R. Gambord, J. Gandhi, D. Gope, T. Grass, B. Hanindhito, A. Hansson, S. Haria, A. Harris, T. Hayes, A. Herrera, M. Horsnell, S. A. R. Jafri, R. Jagtap, H. Jang, R. Jeyapaul, T. M. Jones, M. Jung, S. Kannoth, H. Khaleghzadeh, Y. Kodama, T. Krishna, T. Marinelli, C. Menard, A. Mondelli, T. M"uck, O. Naji, K. Nathella, H. Nguyen, N. Nikoleris, L. E. Olson, M. S. Orr, B. Pham, P. Prieto, T. Reddy, A. Roelke, M. Samani, A. Sandberg, J. Setoain, B. Shingarov, M. D. Sinclair, T. Ta, R. Thakur, G. Travaglini, M. Upton, N. Vaish, I. Vougioukas, Z. Wang, N. Wehn, C. Weis, D. A. Wood, H. Yoon, and 'Eder F. Zulian, "The gem5 Simulator: Version 20.0+," *CoRR*, vol. abs/2007.03152, Jul. 2020. [Online]. Available: https://arxiv.org/abs/2007.03152

[34] P. Messina, "The Exascale Computing Project," *Computing in Science and Engineering*, vol. 19, no. 3, pp. 63–67, May 2017.

[35] H. Patil, R. Cohn, M. Charney, R. Kapoor, A. Sun, and A. Karunanidhi, "Pinpointing Representative Portions of Large Intel Itanium Programs with Dynamic Instrumentation," in *Proceedings of the 37th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO 37. USA: IEEE Computer Society, Dec. 2004, pp. 81–92.

[36] E. Perelman, J. Lau, H. Patil, A. Jaleel, G. Hamerly, and B. Calder, "Cross Binary Simulation Points," in *2007 IEEE International Symposium on Performance Analysis of Systems & Software*, Apr. 2007, pp. 179–189.

[37] Z. Qiu, "The LoopPoint Methodology in the gem5 Simulator," 2023, presentation slides, gem5 Workshop at International Symposium on Computer Architecture (ISCA 2023). [Online]. Available: https://www.gem5.org/assets/files/workshop-isca-2023/slides/looppoint-in-the-gem5-simulator.pdf

[38] R. Rahman, D. Owen, and J. You, "Tracking Large-Scale AI Models," 2024, accessed: 2024-10-18. [Online]. Available: https://epochai.org/blog/tracking-large-scale-ai-models

[39] A. Sabu, H. Patil, W. Heirman, and T. E. Carlson, "LoopPoint: Checkpoint-driven Sampled Simulation for Multi-threaded Applications," in *2022 IEEE International*

Symposium on High-Performance Computer Architecture (HPCA), Apr. 2022, pp. 604–618.

[40] A. Sandberg, N. Nikoleris, T. E. Carlson, E. Hagersten, S. Kaxiras, and D. Black-Schaffer, "Full Speed Ahead: Detailed Architectural Simulation at Near-Native Speed," in *2015 IEEE International Symposium on Workload Characterization*, Oct. 2015, pp. 183–192.

[41] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically characterizing large scale program behavior," in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS X. New York, NY, USA: Association for Computing Machinery, Oct. 2002, pp. 45–57.

[42] Standard Performance Evaluation Corporation (SPEC), "SPEC CPU2000 Benchmark Suite," https://www.spec.org/cpu2000/, 2000, [Online; accessed 21-Oct-2025].

[43] Standard Performance Evaluation Corporation (SPEC), "SPEC CPU2006 Benchmark Suite," https://www.spec.org/cpu2006/, 2006, [Online; accessed 21-Oct-2025].

[44] Standard Performance Evaluation Corporation (SPEC), "SPEC CPUv8 Benchmark Suite," https://www.spec.org/cpuv8/, 2025.

[45] Vertical Research Group, "microbench: Extremely Simple Microbenchmarks," 2018, MIT License. Accessed 2025-10-21. [Online]. Available: https://github.com/VerticalResearchGroup/microbench

[46] Y. Wang, G. M. Stocks, W. A. Shelton, D. M. C. Nicholson, W. M. Temmerman, and Z. Szotek, "Order-N multiple scattering approach to electronic structure calculations," *Phys. Rev. Lett.*, vol. 75, pp. 2867–2870, Oct. 1995.

[47] V. M. Weaver and S. A. McKee, "Using Dynamic Binary Instrumentation to Generate Multi-platform SimPoints: Methodology and Accuracy," in *High Performance Embedded Architectures and Compilers*, P. Stenström, M. Dubois, M. Katevenis, R. Gupta, and T. Ungerer, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, Jan. 2008, pp. 305–319.

[48] Q. Wu, S. Flolid, S. Song, J. Deng, and L. K. John, "Invited Paper for the Hot Workloads Special Session Hot Regions in SPEC CPU2017," in *2018 IEEE International Symposium on Workload Characterization (IISWC)*, Oct. 2018, pp. 71–77.

[49] R. E. Wunderlich, T. F. Wenisch, B. Falsafi, and J. C. Hoe, "SMARTS: accelerating microarchitecture simulation via rigorous statistical sampling," in *30th Annual International Symposium on Computer Architecture, 2003. Proceedings.*, Jun. 2003, pp. 84–95.

[50] J. Xue, T. Xiong, L. Chao, and R. Xue, "SimPoint+: More Stable, Accurate and Efficient Program Analysis," in *Euro-Par 2025: Parallel Processing*, W. E. Nagel, D. Goehringer, and P. C. Diniz, Eds. Cham: Springer Nature Switzerland, 2026, pp. 3–17.

[51] J. J. Yi, S. V. Kodakara, R. Sendag, D. J. Lilja, and D. M. Hawkins, "Characterizing and comparing prevailing simulation techniques," in *11th International Symposium on High-Performance Computer Architecture*, Feb. 2005, pp. 266–277.

Appendix A

Artifact Appendix

A.1 Abstract

This artifact provides the implementation of Nugget, including its LLVM-based analysis and transformation passes, benchmark harnesses, and automation scripts for the experiments in the paper.

Please find the artifact on GitHub: `https://github.com/darchr/nugget-AE`. The Zenodo link will be added to the GitHub README.

The artifact is organized around three core stages: (1) interval analysis to extract basic-block vectors from NAS Parallel Benchmarks (NPB); (2) sample selection via k-means (and random baselines) to choose representative regions; and (3) Nugget creation and validation, which builds and runs Nugget and naive binaries and computes program-level prediction error.

We provide a Docker-based environment plus a top-level `install.sh` script to encapsulate system dependencies and toolchain setup. Together with the scripts under `nugget-protocol-NPB` and the detailed README, this enables reviewers to reproduce the main trends of the paper (prediction accuracy and runtime overhead) with minimal manual configuration.

A.2 Artifact check-list (meta-information)

- **Algorithm:** k-means-based interval clustering; prediction from sampled regions.

- **Program:** LLVM analysis/transformation passes; NPB-based benchmarks with Nugget instrumentation; automation scripts.
- **Compilation:** Custom LLVM build (with Nugget passes); `clang/opt`; `cmake/make`.
- **Transformations:** LLVM IR analysis; region marking; Nugget code insertion.
- **Binary:** Nugget-enabled and naive NPB and LSMS executables.
- **Model:** k-means clustering over interval basic-block vectors (constructed at evaluation time).
- **Data set:** NAS Parallel Benchmarks; LSMS; generated IR-level CSVs; runtime and prediction CSVs.
- **Run-time environment:** Linux (Ubuntu-like) inside Docker (recommended).
- **Hardware:** Multi-core CPU with hardware performance counters; server-class machine recommended.
- **Execution:** Command-line Python and shell scripts; Makefiles.
- **Metrics:** Runtime; prediction error vs. full-program execution; optional hardware counters.
- **Output:** CSV files for analysis, selections, measurements, and prediction error; log files and timing summaries.
- **How much disk space required (approximately)?:** ~50–100 GB for toolchains, benchmarks, Docker image, and generated data (full runs).
- **How much time is needed to prepare workflow (approximately)?:** ~1–2 hours with Docker on a modern server.
- **How much time is needed to complete experiments (approximately)?:** A few hours for minimal runs (class A, subset of benchmarks); up to days for full-scale on hardware experiments and up to months for simulator experiments.
- **Publicly available?:** Yes (repository / archive as provided with this submission).
- **Workflow automation framework used?:** Custom scripts, CMake scripts, and Makefiles; no external workflow engine.

A.3 Description

A.3.1 How to access

The artifact is provided as a source tree whose root we denote by `[project dir]`. The main components relevant for artifact evaluation are:

- `Docker/`: infrastructure to build and run a container with all system dependencies.
- `llvm-project/`: LLVM with Nugget passes.
- `nugget-protocol-NPB/`: NPB benchmarks and scripts for analysis, sample selection, and Nugget creation/validation.
- `nugget_util/`: helper tools (PAPI, gem5 m5ops, Sniper sim_api, CPU feature detection).
- `gem5-experiment/`: optional scripts for gem5-based experiments.
- Top-level `install.sh`: drives the installation of LLVM, PAPI, and related tools.

The top-level README in the GitHub repository provides the concrete commands; this appendix focuses on the structure and purpose of each step rather than step-by-step instructions.

A.3.2 Hardware dependencies

The experiments are designed for a Linux server-class machine with:

- A recent multi-core CPU (scripts tested on x86_64 and AArch64) with hardware performance counters enabled.
- Enough memory to build LLVM and run NPB/LSMS (we recommend at least 64 GB).
- Sufficient disk space to hold the LLVM build, Docker image, and generated data.

The paper experiments additionally used standard noise-reduction measures (fixed CPU frequency, reduced background activity). These are optional but can help reduce variance.

A.3.3 Software dependencies

The recommended route is to use the provided Docker setup, which bundles:

- Ubuntu 24.04.
- Build tools and libraries needed for LLVM and NPB.
- Python 3 and required Python packages.
- `perf` and PAPI for hardware performance measurement.

A.3.4 Data sets

The only external benchmark suite required is NAS Parallel Benchmarks (NPB) and LSMS, included or fetched by the artifact. All other data (interval analysis CSVs, measurement CSVs, prediction-error CSVs) are generated by the scripts from the code itself and from NPB runs.

A.3.5 Models

Nugget does not rely on pre-trained external models. Instead:

- k-means clustering is run directly on the interval basic-block vectors produced by the analysis scripts;
- the resulting clusters and weights are stored in CSV form and used immediately to drive sampling and prediction.

This design ensures that all modeling steps are visible and reproducible from the code and generated data alone.

A.4 Installation

The installation workflow is intentionally simple and centralized:

1. **Container setup.** Build and run the Docker image from `Docker/`. This fixes the OS-level environment and avoids package-version drift. The Makefile in this directory also exposes a `NUM_CORES_TO_USE` parameter to trade off build speed and memory use.

2. **Toolchain and helper installation.** Inside the container, run `install.sh` from `[project dir]`. This script:
   - builds LLVM with Nugget passes;
   - installs PAPI in a known location;
   - sets up helper tools (e.g., CPU feature checker, optional simulator hooks);
   - prints the environment variables that must be exported to use this toolchain.
3. **Environment configuration.** The script prints `export` commands at the end. Copy-pasting these commands into the shell (or into a small script to source) ensures that all subsequent steps use the correct LLVM and PAPI paths.

All low-level commands, including package lists and exact `make` invocations, are documented in the README; this appendix only captures the intent and structure of the workflow.

A.5 Experiment workflow

The experiment workflow is structured to mirror the conceptual pipeline of Nugget:

1. **Preparation and interval analysis.** Scripts in `nugget-protocol-NPB` compile NPB using the Nugget-enabled LLVM toolchain and run interval analysis. The result is a set of CSV files containing basic-block vectors and timing information, organized per benchmark, input class, and thread count. This stage bridges the gap between high-level benchmark code and the IR-level representation used by Nugget.
2. **Sample selection.** A dedicated script runs k-means on the basic-block vectors to identify representative regions and optionally creates random baselines. The output is a set of selections/markers, again stored as CSV and configuration files. This stage captures the core Nugget sampling logic in a reproducible and inspectable way.
3. **Nugget creation and validation.** A third script uses the selections to build Nugget and naive binaries, executes them, and aggregates the runtimes into `measurements.csv`.

It then computes program-level runtime predictions based on the sampled regions and emits `prediction-error.csv`. This stage directly corresponds to the evaluation part of the paper and provides the main numbers needed to confirm the reported trends.

4. **Optional simulator experiments.** For gem5 and Sniper, helper scripts and utilities in `nugget_util` and `gem5-experiment` provide the glue to run Nugget inside simulators. These are not required to validate the main claims but are included for completeness and extension.

Each stage is driven by a small number of top-level Python scripts with clearly documented command-line options. The README links these scripts directly to the corresponding sections of the paper.

A.6 Evaluation and expected results

The artifact is designed so that each stage produces a small set of key files that map directly to the paper's figures and tables:

- **Interval analysis:** Per-benchmark directories under `ae-experiments/analysis` contain:
  - `analysis-output.csv` (IR-level basic-block vectors and related data);
  - `execution_time.txt` (time spent in analysis).

  These files support claims about analysis cost and IR-level behavior.

- **Sample selection:** The selection step produces cluster assignments and markers under `ae-experiments/`, making the sampling decisions explicit and inspectable.

- **Nugget creation and validation:** `measurements.csv` and `prediction-error.csv` summarize:
  - baseline runtimes (full naive runs);
  - Nugget runtimes (sampled runs);

– prediction errors for both k-means-based and random sampling.

These are the primary artifacts for validating the paper's quantitative claims.

Because the artifact is evaluated on potentially different hardware and under different levels of system noise, we do not expect identical results. Instead, success is defined as:

- the complete pipeline runs to completion following the documented workflow;

A.7 Experiment customization

The artifact intentionally exposes key parameters as command-line options:

- input class (NPB size), benchmark subset, and thread count;
- number of clusters (`k`) and grace percentage used in selection and Nugget creation;
- number and composition of hardware events (via PAPI helpers);
- optional simulator-specific configuration (gem5 ISAs, cross-compilers, Sniper hooks).

This makes it straightforward to:

- scale experiments up or down depending on available resources;
- explore trade-offs between sampling cost and prediction accuracy;
- port the methodology to different machines while retaining the same high-level workflow.